

Task3: Secure Coding Review

TITLE:

Secure Coding Review of a Python Login System

By: Meghana Ankam

ID: CA/DF1/82501

Objective:

To identify security vulnerabilities in a Python-based login system and implement secure coding practices to mitigate them.

Tools Used:

- Programming Language: Python
- IDE: Visual Studio Code
- Method: Manual Code Review

Description of the Application:

The application is a simple Python login system that accepts a username and password and checks them against stored credentials for authentication. It is used to demonstrate basic login functionality and identify security issues during code review.

Vulnerable Code:

```
username = input("Enter username: ")
password = input("Enter password: ")
query = "SELECT * FROM users WHERE username='"
+ username + "' AND password='" + password + "'"
print("Executing Query:", query)
if username == "admin" and password ==
"admin123":
    print("Login Successful")
else:
    print("Login Failed").
```

Identified Vulnerabilities:

SQL Injection

User input is directly added to the query, allowing attackers to manipulate it.

Plain Text Password

Passwords are stored and compared in plain text.

No Input Validation

The system accepts any type of input without checking.

Weak Authentication

No encryption or advanced security mechanisms are used.

Attack Demonstration:

Input:

Username: ' OR '1'='1

Password: anything

Generated Query:

```
SELECT * FROM users WHERE username="" OR '1'='1' AND  
password='anything'
```

This condition is always true, showing a SQL Injection vulnerability.

Secure Code:

```
import hashlib

stored_username = "admin"

stored_password_hash =
hashlib.sha256("admin123".encode()).hexdigest()

username = input("Enter username: ")
password = input("Enter password: ")

password_hash =
hashlib.sha256(password.encode()).hexdigest()

if username == stored_username and password_hash ==
stored_password_hash:
    print("Login Successful")
else:
    print("Login Failed")
```

Improvements Made:

- Implemented password hashing using SHA-256
- Removed direct use of user input in queries
- Improved authentication mechanism.

Comparison:

Feature	Vulnerable Code	Secure Code
SQL Injection	Yes	No
Password Storage	Plain Text	Hashed
Security Level	Low	High

Conclusion:

The secure coding review helped identify critical vulnerabilities in the login system. By applying secure coding practices such as hashing and proper input handling, the application was made more secure and resistant to common attac